



PORTFOLIO

FrontEnd Developer

Han Yubin
한유빈

Han Yubin,
A Developers who are curious
about problem solving
and create creative solutions

FrontEnd Developers Portfolio

Contents.

Han Yubin,
A Developers who are curious
about problem solving
and create creative solutions

➔ 01 자기소개
Introduction

➔ 03 프로젝트 경험
Project Experience

➔ 02 핵심역량
Competencies

➔ 04 컨택문의
Contact



Introduction.

I am a developer

소통하는 개발자 **한유빈**입니다

사용자 경험 향상과 협업 능력을 최우선으로 생각하는 프론트엔드 개발자, 한유빈입니다.
한 번 개발하고 끝내는 게 아니라 지속적으로 개선하고 최적화하는 것을 좋아합니다!



한 유 빈 / Han Yubin 2000.01.21

Graduation.

- ✓ 2025. 02 광운대학교 소프트웨어학부 학사 졸업
소프트웨어 세부전공
4.27 / 4.5 학점

Experience.

- ✓ 2023. 01 - 2023.06 **티에프엠 주식회사 개발 인턴**
대표 웹사이트 UI 개편 및 반응형 전환
K-ESG 설문조사 기능 및 리포팅 서비스 화면 개발
실무 워크플로우 경험
- ✓ 2025. 02 - 2025. 08 **프로그래머스 데브코스: 클라우드 기반 프론트엔드 엔지니어링**
서비스 기획부터 배포까지 전 과정 참여
사용자 경험 개선을 위한 UI/UX 디자인 설계
코드 리뷰 및 Jira 스프린트 운영을 통한 팀 단위 협업 경험

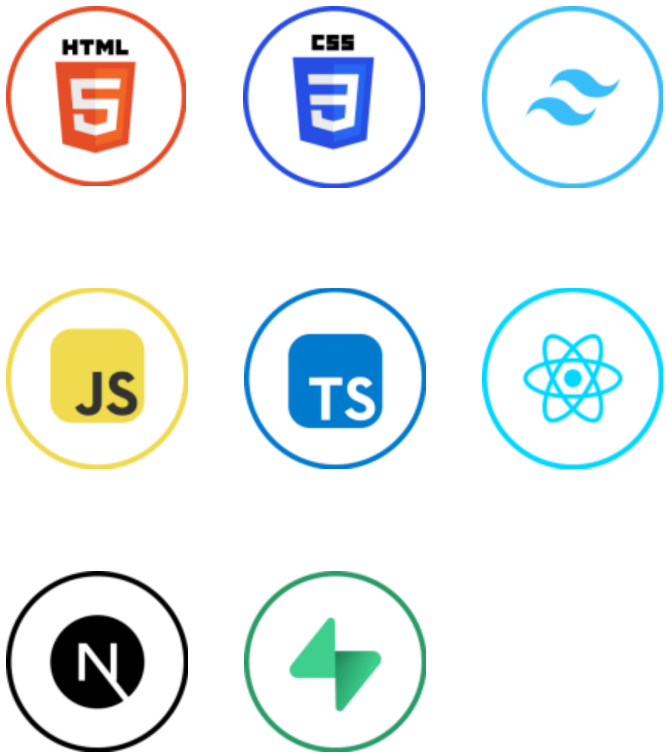
Awards.

- 우수 프로젝트 선정
- Dean's List 성적 우수 학과장상
- 글로벌인재트랙 일본어 영역 인증

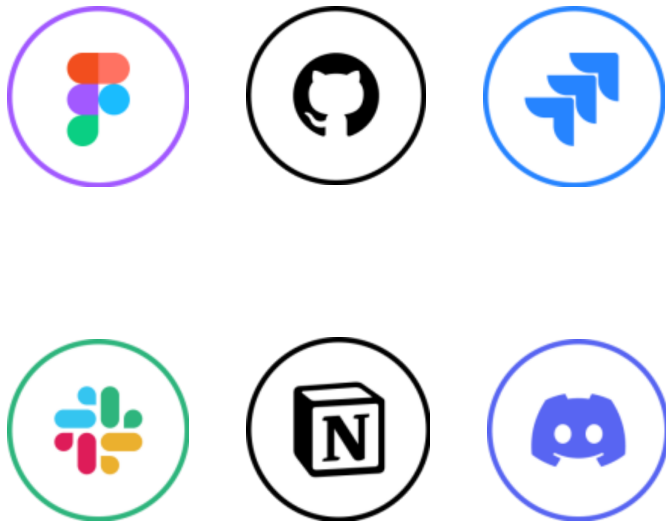


Competencies.

Yubin's Skills



Skills



Tools

리더십	소통	문제해결
책임감	질문	지식공유

Teamwork



Project Experience.

Project 01

올인원 스터디 모집 및 관리 플랫폼

Wibby는 단순한 스터디 매칭을 넘어 스터디의 **모집 - 참여 - 운영 - 종료** 전 과정을 통합적으로 지원하는 올인원 스터디 관리 플랫폼입니다. 기존 스터디 서비스들이 모임 개설과 매칭 기능에만 국한된 것과 달리, 스터디 생성 이후의 **지속적인 운영과 협업**에 중점을 두었습니다. 또한 팀원 간 실시간 소통, 캘린더 기반 일정 관리, 자기 PR 기반의 팀 매칭 등을 통해 자율적이고 효율적인 스터디 문화를 지향합니다.

✅ **Period** 2025년 6월 - 2025년 7월

✅ Performance

- 사용자 수 : 배포 후 **83명의 사용자 확보**
- 고객 만족도 : 사용자 응답 결과 **90% 이상의 만족도 기록**
- 시장 반응 : 투표 결과에서 '**우수 프로젝트**'로 선정

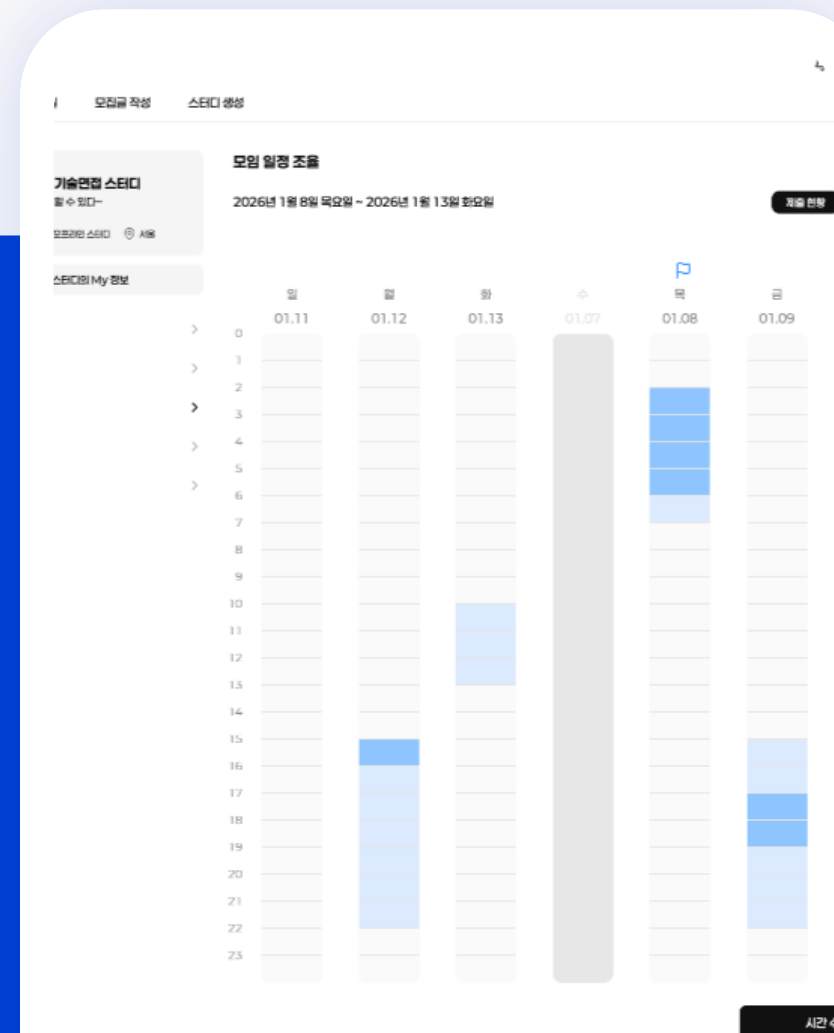
✅ Participation

총 9명 (프론트엔드 개발자 4명, 백엔드 개발자 5명)

✅ Contribution

기획 30% / UIUX 설계 40% / 프론트엔드 개발 40% / 배포 100%

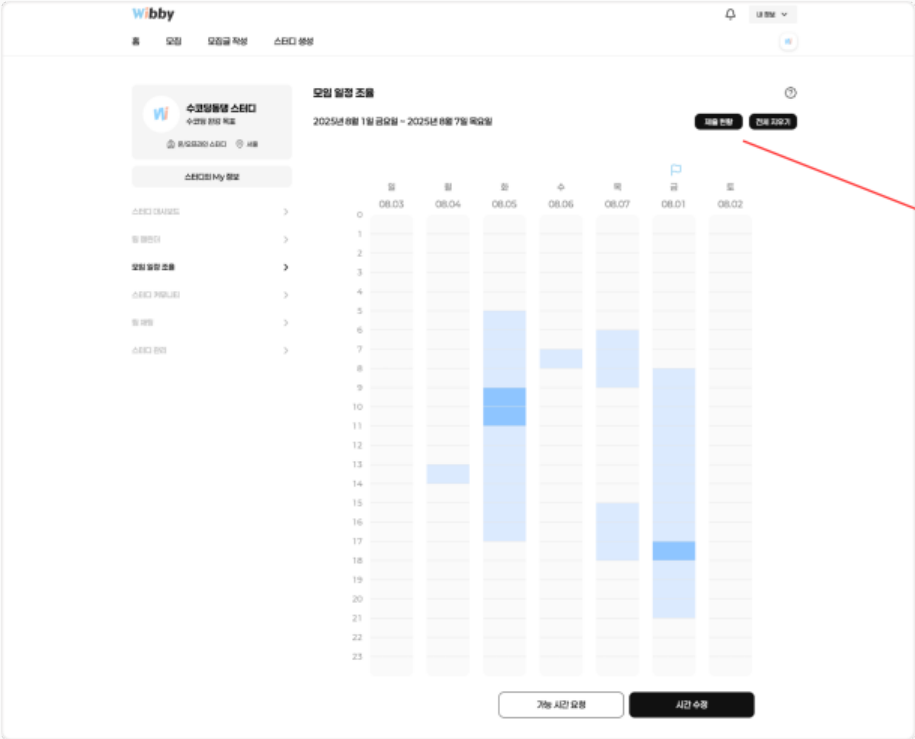
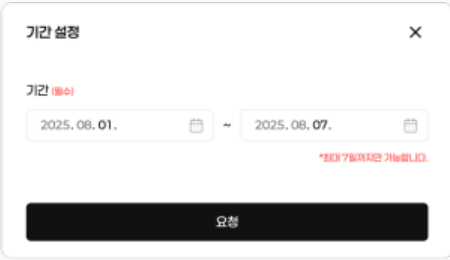
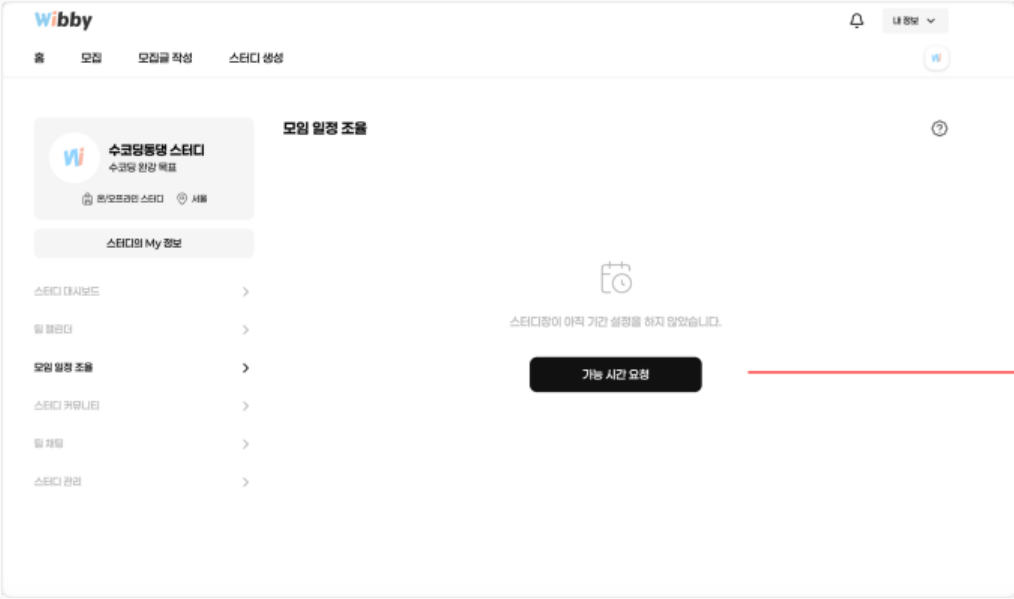
#TypeScript #Next.js #TailwindCSS #Zustand



모임 일정 조율

스터디원 매너 평가

Project 01 모임 일정 조율 리팩토링



✓ 상황

배열로 시간 데이터를 주고받는 방식으로 서버와 통신했지만, 시간을 선택하고 제출할 때 **로딩 시간이 길어** 사용자 경험을 해치는 상황입니다.

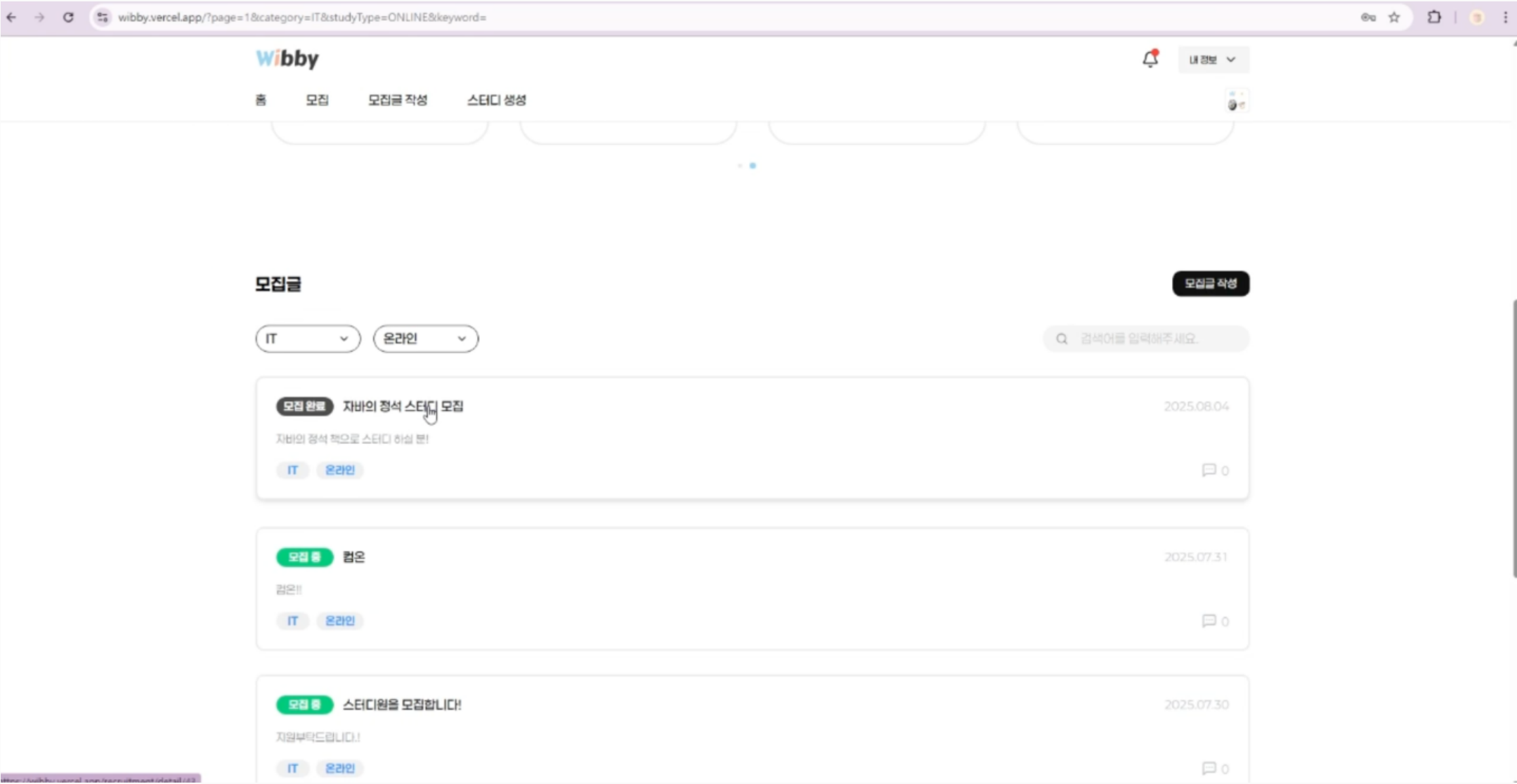
✓ 실행

BitMask 방식을 적용하여 시간 데이터를 처리하였습니다. 이유는 아래와 같습니다.

- **수행 시간 단축** : 비트 연산이기 때문에 $O(1)$ 에 구현되는 것이 많습니다. 비트의 개수 만큼 원소를 다룰 수 있기 때문에 연산 횟수가 늘어날수록 차이가 매우 커지게 됩니다.
- **메모리 사용량 절약** : 하나의 정수로 매우 많은 경우의 수를 표현할 수 있기 때문에 메모리 측면에서 효율적입니다. 비트가 10개인 경우에는 2^{10} 가지의 경우를 10bit 이진수 하나로 표현 가능합니다.

✓ 결과

선택된 시간을 BitMask 연산으로 처리하여 개선한 결과, 로딩 속도가 **약 8초**나 빨라졌습니다.



wibby.vercel.app/?page=1&category=IT&studyType=ONLINE&keyword=

✓ 상황

스터디 모집글 목록에서 새로고침을 하거나 링크를 공유할 시 **사용자 의도와 다르게 필터링 옵션이 초기화**되는 불편한 상황입니다.

✓ 실행

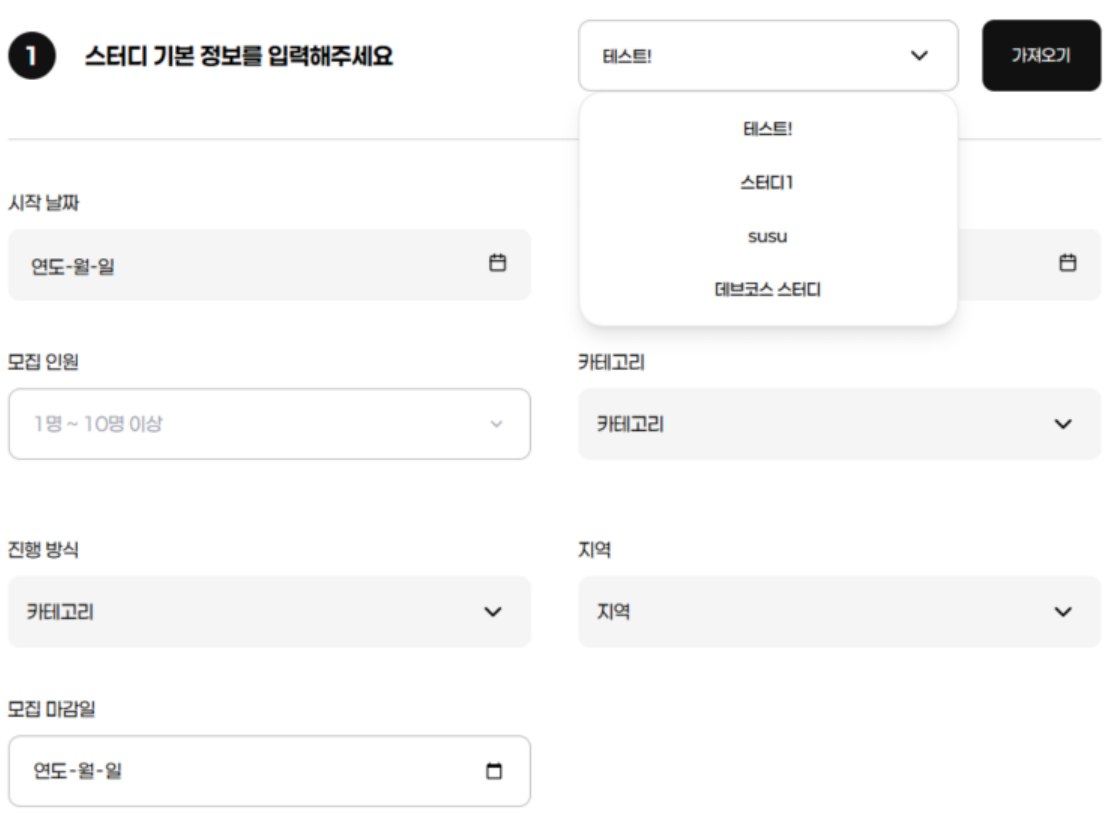
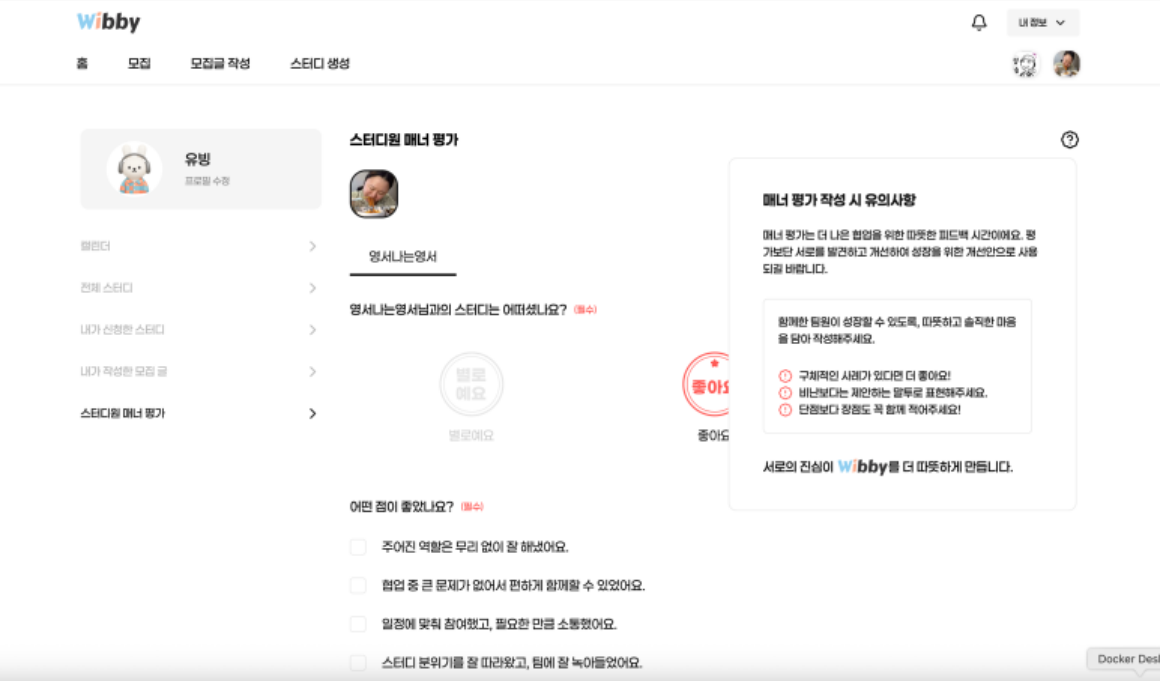
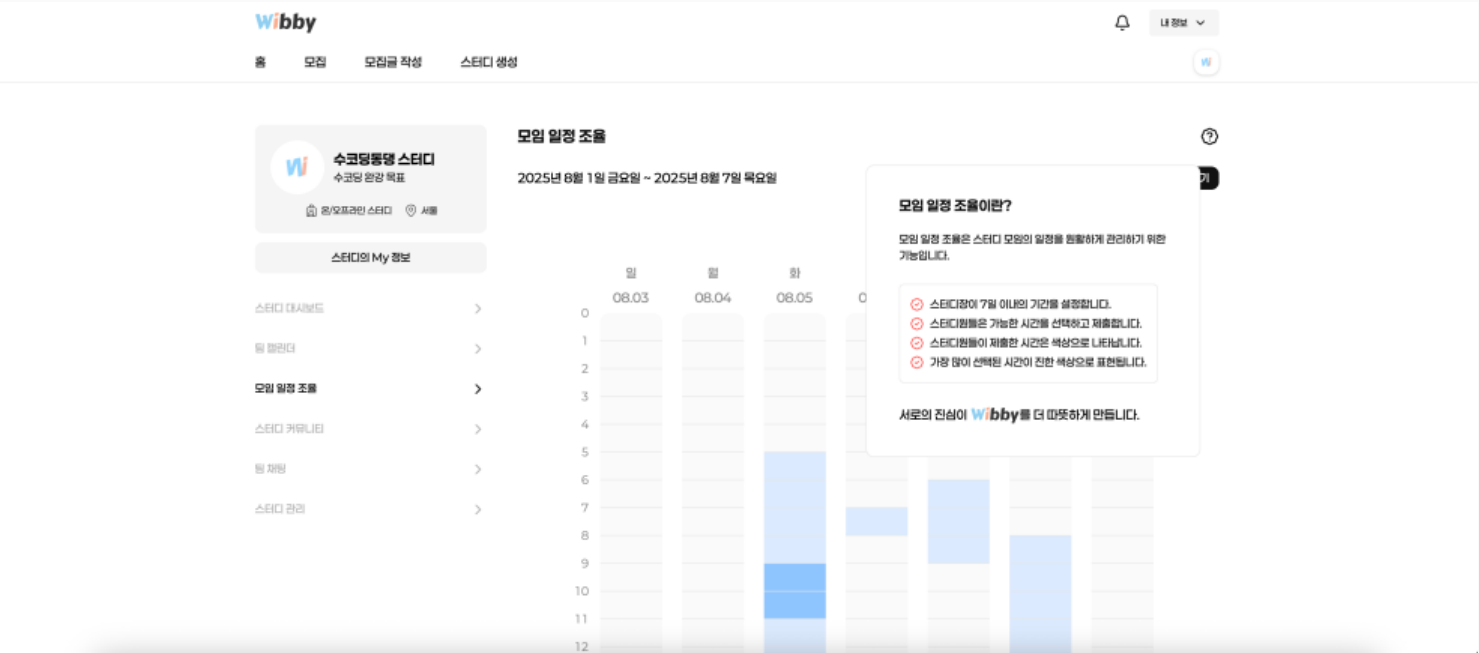
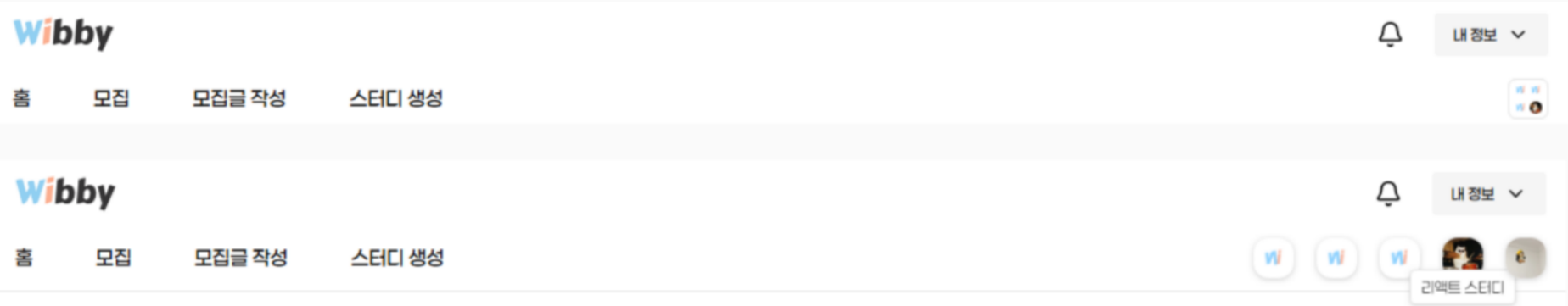
Query String을 활용하여 URL 상태를 관리했습니다. 이유는 아래와 같습니다.

- **사용자 경험 향상** : 스터디 모집글을 보고 타인에게 공유하는 일은 매우 자연스러운 경험이므로 사용자의 의도대로 필터링된 링크를 공유할 수 있습니다. 또한, 주소창에 정보를 남기면 사용자에게 되돌아올 수 있는 방법을 열어줄 수 있습니다.
- **서버 영향 고려** : GET 요청은 서버의 상태를 변경하지 않으므로, 필터링 옵션을 Request Body에 넣어 POST 요청을 하는 것보다 Query String에 넣어 GET 요청을 하는 것이 더 좋다고 판단했습니다.

✓ 결과

Query String을 활용함으로써 새로고침 및 링크 공유, 뒤로가기 등의 상황에서 발생하는 불편함을 해결할 수 있었습니다.

Project 01 UX 개선



☑ 서브 헤더 스터디 목록

현재 진행 중인 스터디가 있다면 해당 스터디 공간으로 이동할 수 있는 버튼이 서브 헤더에 생깁니다. 만약 **진행 중인 스터디가 여러 개인 사용자**라면 각각의 스터디 공간으로 이동하는 버튼이 서브 헤더에 줄줄이 나열되면서 화면상 깔끔하지 못하고 불편할 수 있다고 생각했습니다. 따라서 일정 개수 이상으로 늘어난다면 자동으로 **폴더 형식으로 변환**되게 하여 마우스 호버 시 각각의 공간을 확인할 수 있도록 구현했습니다.

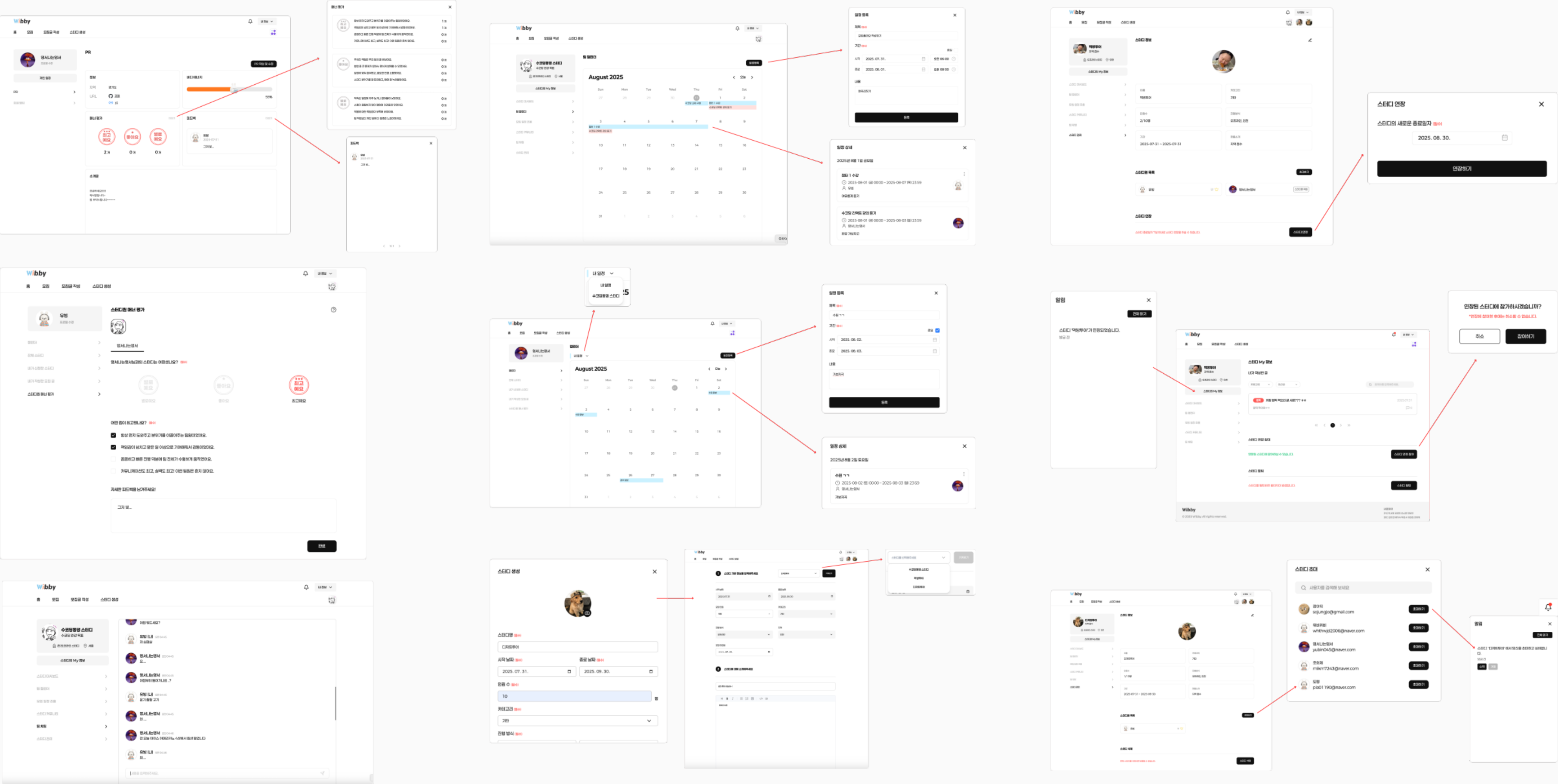
☑ 도움말 툴팁

스터디원 매너 평가 기능과 모임 일정 조율 기능은 **사용 방법이 다소 생소**할 수 있다고 생각하여 사용자에게 **간단한 설명과 이용 방법**을 알려주는 도움말 툴팁을 추가했습니다. 이를 활용하여 사용자가 어떤 기능인지 빠르게 이해하고 더 나은 서비스 경험을 얻을 수 있도록 하였습니다.

☑ 스터디 가져오기

이미 스터디 생성 시 입력했던 정보들을 모집글 작성 시에도 중복으로 또 입력해야 한다면 상당히 번거로울 수 있다고 생각했습니다. 따라서 모집글을 작성할 때는 **기존 스터디 정보를 그대로 불러와** 자동으로 채워지도록 하여 불편함을 개선했습니다.

Project 01 기타 UI



Project Experience.

Project 02

낙서를 통해 소통하는 그림 퀴즈 게임

Cat's Paw는 고양이 손을 빌린 듯한 귀여운 낙서를 그리고 맞히며 소통하는 웹 기반 게임입니다.

기존 유사 서비스들과 달리, 모든 유저가 **동시에** 그림을 그리고 정답을 맞히는 구조로 기다릴 틈 없이 몰입감 있게 게임을 즐길 수 있습니다.

또한 혼자서도 즐길 수 있는 **AI 싱글모드**를 제공하며, 마지막에 받은 점수를 통해 **랭킹** 확인도 가능합니다.

✅ **Period** 2025년 5월 - 2025년 6월

✅ Performance

- 사용자 수 : 배포 후 **40명의 사용자 확보**
- 고객 만족도 : 사용자 응답 결과 **98% 이상의 만족도 기록**
- 시장 반응 : 실사용자에게서 '**시간 가는 줄 모르고 하게 된다**' 등 긍정적 평가

✅ Participation

총 4명 (프론트엔드 개발자 4명)

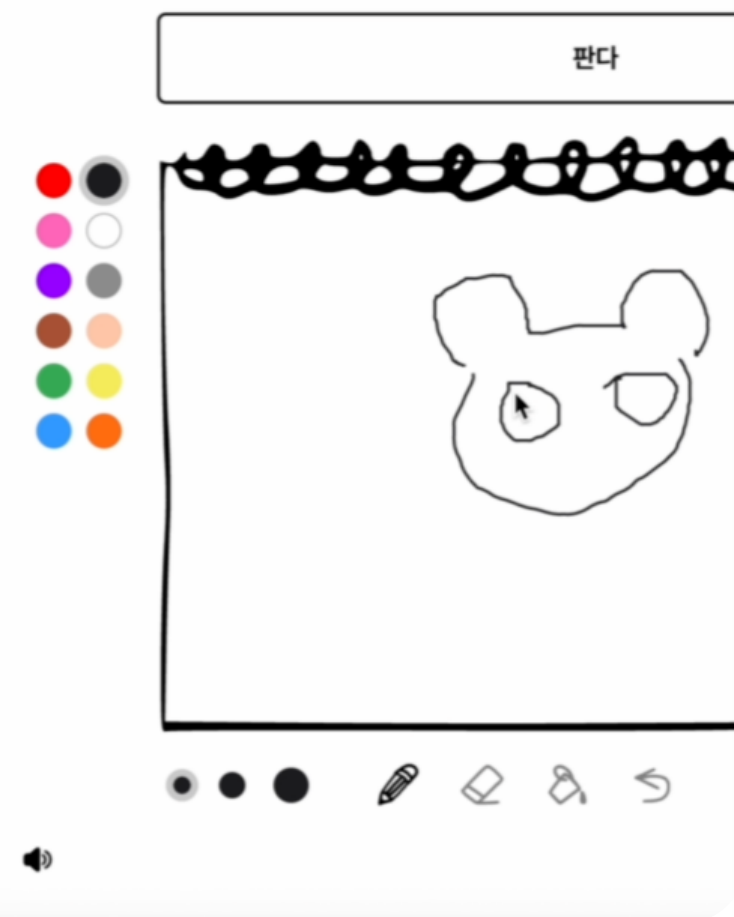
✅ Contribution

기획 30% / UIUX 설계 30% / 프론트엔드 개발 40% / 배포 100%

#TypeScript #React #TailwindCSS #Zustand #Supabase



같이 할 사람



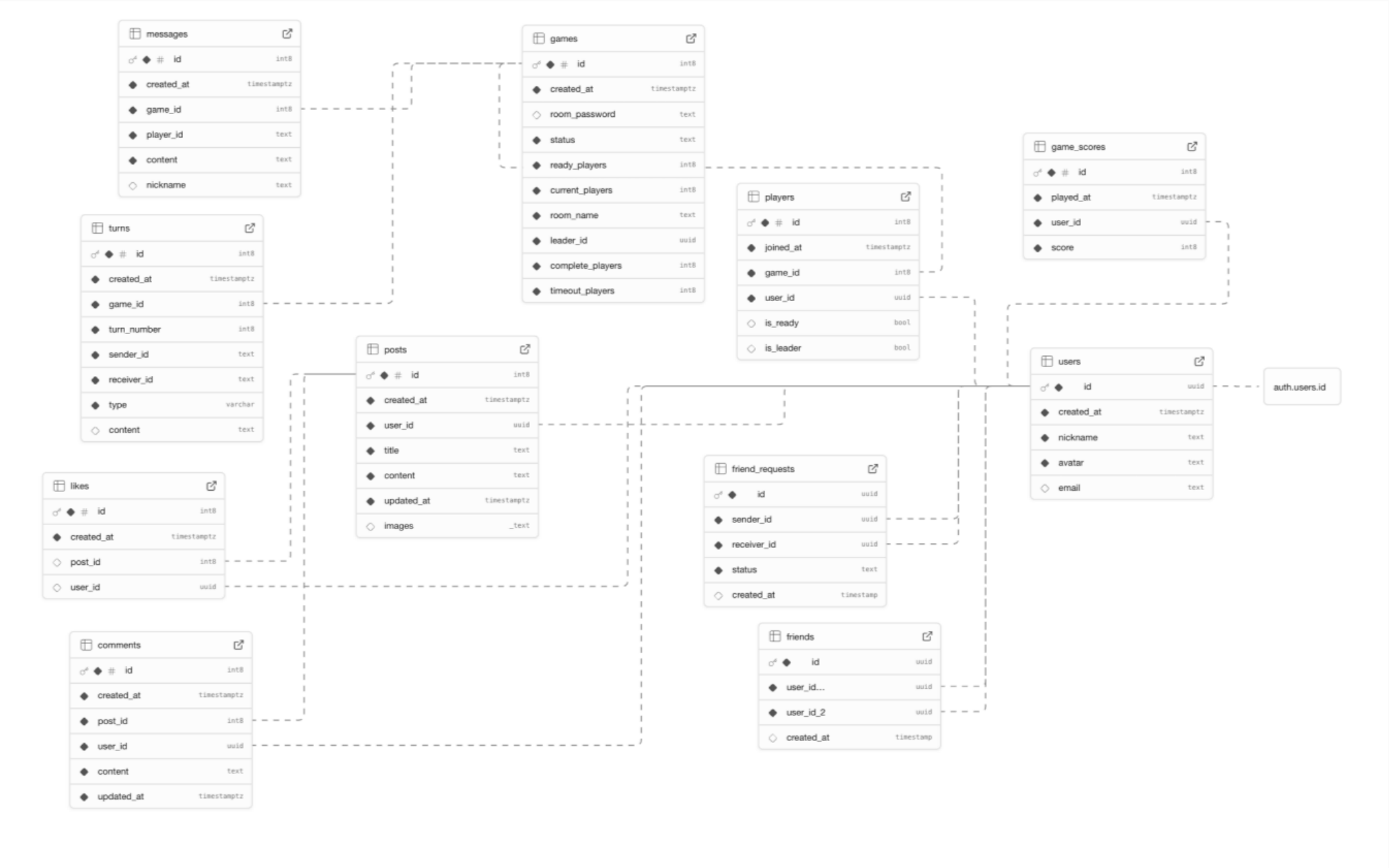
멀티모드

슬 모드



AI 싱글모드

Project 02 데이터 스키마 설계



✓ 상황

게임 룸 - 플레이어 - 턴 - 제시어 간 관계를 고려하여 Supabase 기반 데이터 스키마를 설계해야 하는 상황입니다. 또한, 게임 뿐만 아니라 **시 모드**의 **랭킹 점수**, **사용자 정보**, **친구**, **커뮤니티** 등의 데이터도 관리해야 하는 상황입니다.

✓ 실행

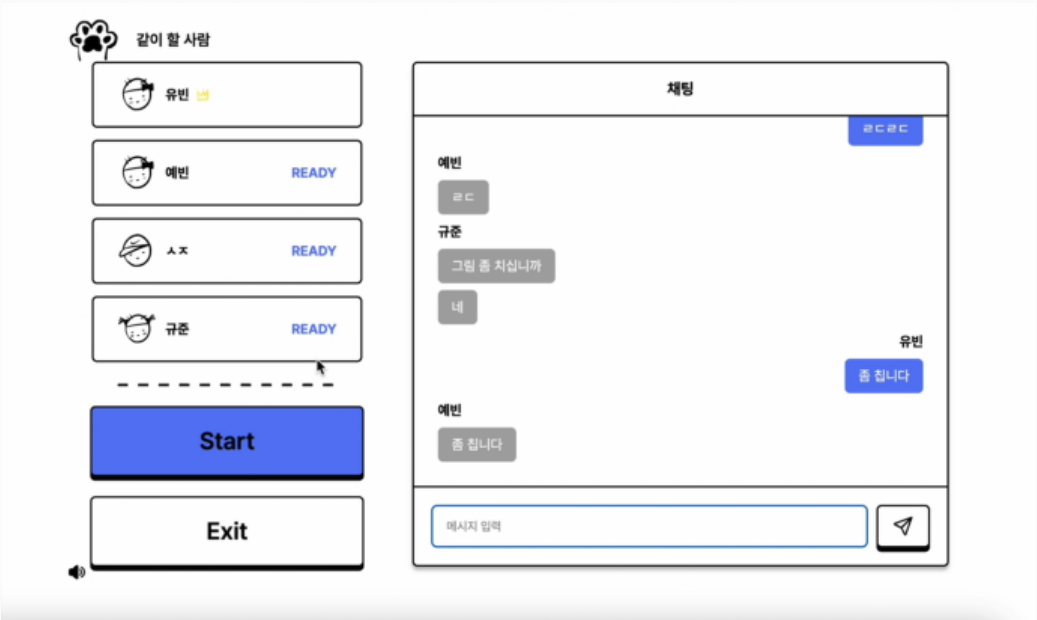
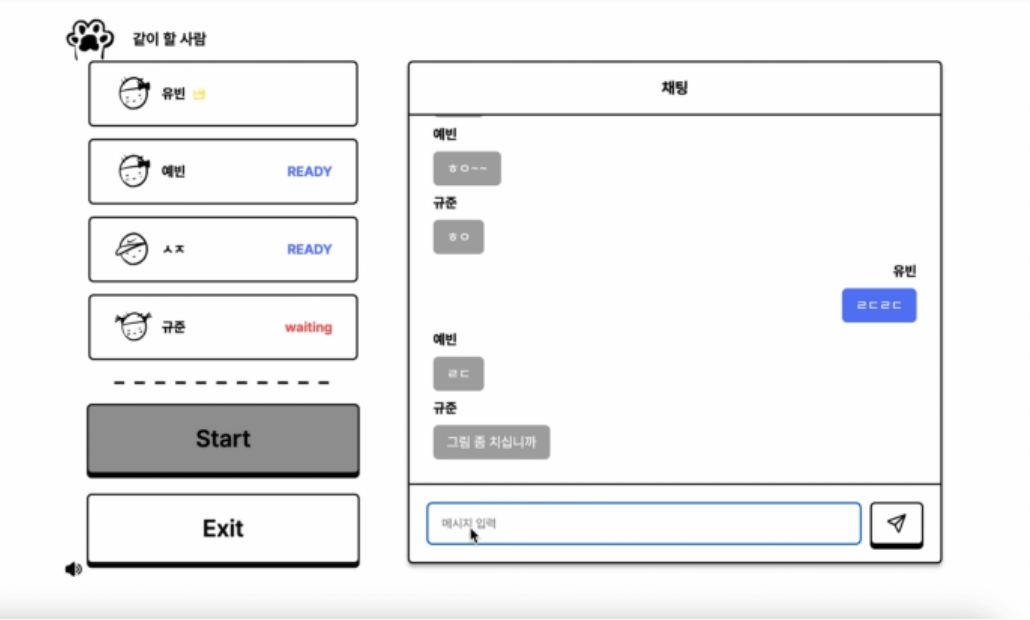
왼쪽과 같이 설계하여 관리했습니다. 이유는 아래와 같습니다.

- 게임 상태 관리와 실시간 동기화를 고려한 구조 분리 : 게임의 각 라운드를 독립적인 **turn** 단위로 저장하여 실시간 구독(Realtime)과 결과 재구성이 모두 가능하도록 설계했습니다. sender/receiver를 명확히 분리함으로써 **자기 데이터 재수신 문제를 방지**했고, Supabase Realtime 필터링 구조에 최적화된 설계를 적용했습니다.
- 게임을 넘어서 커뮤니티/소셜 확장을 고려한 구조 : 단순 게임 세션 저장 구조를 넘어, **사용자 간 친구 관계, 게시판, 점수 시스템**까지 확장 가능한 소셜 플랫폼 구조로 설계했습니다. 인증과 프로필을 분리하여 Supabase 권장 아키텍처를 따랐으며, 게임 중심 서비스에서 커뮤니티 서비스로도 확장할 수 있도록 설계했습니다.

✓ 결과

라운드, 배정, 제출을 분리한 구조 덕분에 게임의 공정한 **랜덤 배정**과 **라운드 흐름 제어**를 DB 레벨에서 보장할 수 있게 되었고, Supabase Realtime 필터링과 확장성을 고려해 이벤트 단위로 구조화하여 실시간 동기화와 성능을 동시에 확보할 수 있었습니다.

Project 02 멀티모드 진행 로직



✓ 모드 선택 및 BGM 설정

모드 선택 화면을 게임 상태 전환의 시작점으로 설계했습니다.

라우팅 기반으로 BGM이 자동 전환되도록 구현하고, Audio 객체를 직접 관리해 안정성과 메모리 누수를 방지했습니다.

또한 커스텀 훅을 통해 게임 시작 전 상태를 정리하고, 카운트다운 전환으로 UX를 개선했습니다.

✓ 게임방 생성 및 입장

Supabase Realtime을 활용해 게임방 목록을 **실시간 동기화**하고, 방 상태에 따른 사전 검증 로직으로 안정적인 입장 흐름을 설계했습니다.

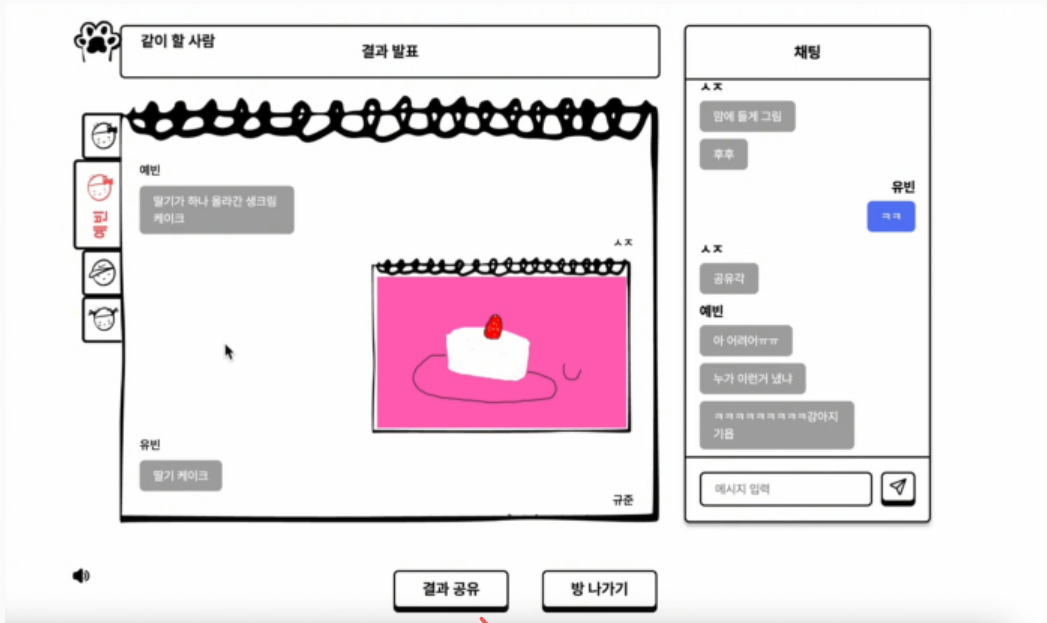
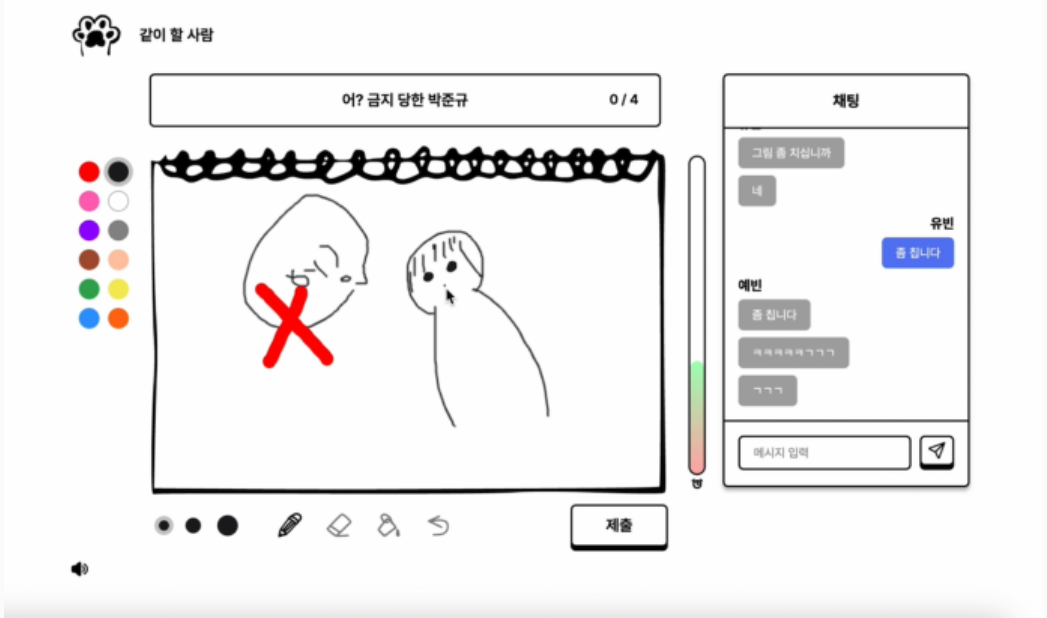
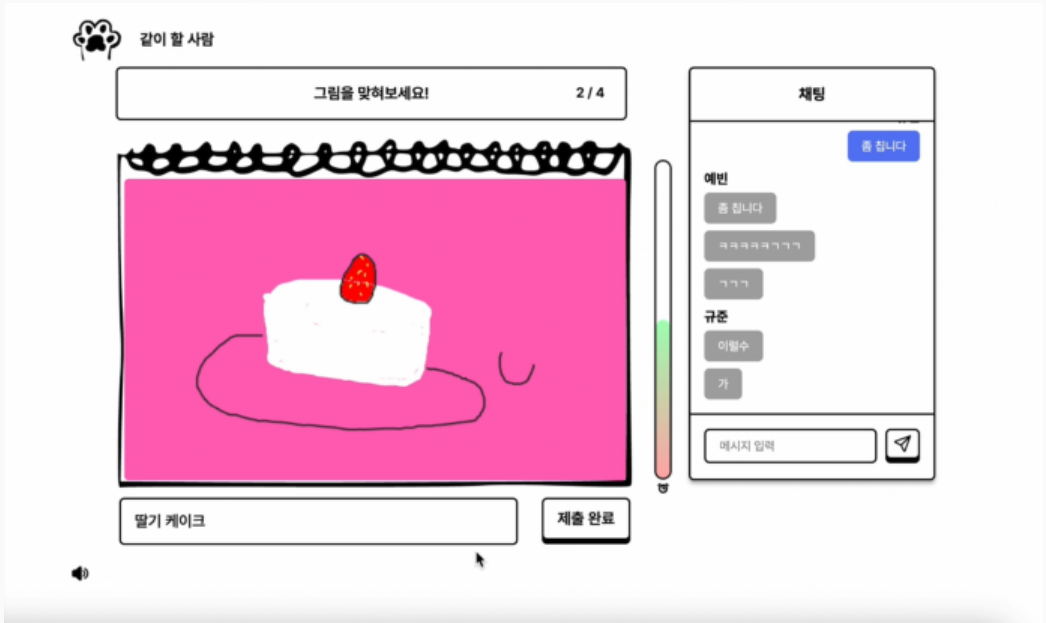
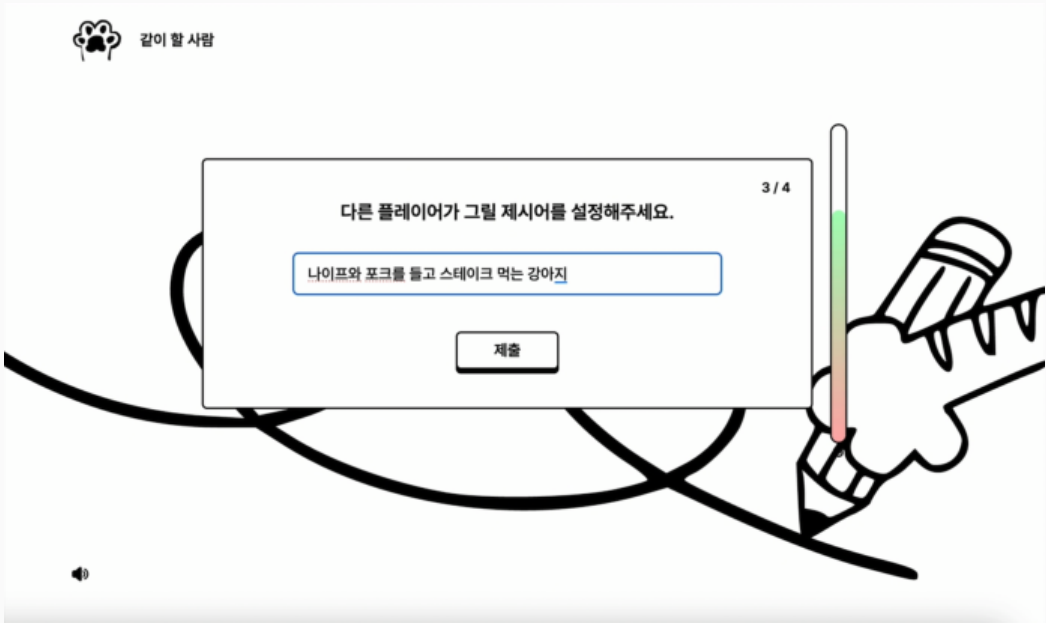
입장 시 players 및 games 테이블을 순차적으로 갱신하고 전역 상태를 초기화하여 **게임 세션의 일관성을 보장**했으며, debounce를 적용해 중복 요청을 방지했습니다.

✓ 게임 대기방 준비 상태 및 실시간 채팅

Supabase Realtime을 활용해 players 테이블을 game_id 단위로 구독하고, 준비 상태 및 인원 변화를 실시간 동기화했습니다.

게임 시작은 **leader만 턴 생성 API를 호출**하도록 설계해 중복 실행을 방지했으며, 채팅은 **Broadcast와 DB 저장을 분리**하여 실시간성과 영속성을 동시에 확보했습니다.

Project 02 멀티모드 진행 로직



✓ 제시어 설정

제시어 입력 결과를 turns 테이블의 turn_number와 sender_id 기준으로 저장하여 각 플레이어의 턴 진행 상태를 관리했습니다. complete_players 카운트를 활용해 모든 플레이어 제출 여부를 판단하고, Supabase Realtime을 통해 **다음 턴으로 자동 전환**되도록 설계했습니다. 또한 **턴별 타이머를 도입**하여 제출하지 않은 플레이어가 있어도 게임이 멈추지 않도록 처리했습니다.

✓ 드로잉 및 제시어 예측

React Konva 기반 Canvas 드로잉 시스템을 구현하여 펜, 지우개, 페인트, 색상 선택, Undo 기능 등을 제공했습니다. Canvas 이미지는 Base64 데이터를 Blob으로 변환하여 **Supabase Storage에 업로드**하고, 생성된 Public URL을 turns 테이블에 저장하는 방식으로 설계했습니다. 또한 turns 테이블의 sender_id와 receiver_id를 활용하여 플레이어 간 그림과 제시어가 **순환 전달되는 턴 기반 데이터 구조**를 구현했습니다.

✓ 결과 화면 공유

게임 종료 후 turns 데이터를 재구성하여 **플레이어별 그림과 제시어 흐름을 시각적으로 보여주는 결과 화면**을 구현했습니다. 또한 html2canvas를 활용해 **결과 화면을 이미지로 캡처**하고 Supabase Storage에 업로드하여 게임 결과를 게시글 형태로 공유할 수 있도록 설계했습니다.

Project 02 Edge Function 기반 멀티플레이 설계

```
const { game, player } = await req.json()

const { data: newGameRoom, error: gameError } = await supabase
  .from('games')
  .insert(game)
  .select()
  .single()
```

```
const playerWithGameId = {
  ...player,
  game_id: newGameRoom.id,
  user_id: newGameRoom.leader_id,
  is_ready: true,
  is_leader: true,
}

const { data: newPlayer, error: playerError } = await supabase
  .from('players')
  .insert(playerWithGameId)
  .select()
```

```
const { game_id } = await req.json();

const { data: players, error: playerError } = await supabase
  .from("players")
  .select("user_id")
  .eq("game_id", game_id);
```

```
const player_ids = players.map((p) => p.user_id);
const shuffled = [...player_ids].sort(() => Math.random() - 0.5);

for (let i = 0; i < shuffled.length; i++) {
  const turns = shuffled.map((sender, idx) => {
    const receiver = shuffled[(idx + 1) % shuffled.length];
    return {
      game_id,
      turn_number: i + 1,
      sender_id: sender,
      receiver_id: receiver,
      type: i % 2 === 0 ? 'WORD' : 'DRAWING',
    };
  });
}
```

✓ createRoomWithLeader - 방 생성 트랜잭션 분리

여러 테이블 데이터를 동시에 생성해야 했기 때문에 Edge Function을 사용하여 서버에서 원자적으로 처리하도록 설계했습니다.

- games 테이블에 방 생성
- players 테이블에 리더 자동 추가
- 리더는 is_ready = true
- is_leader = true 설정

✓ createTurns - 게임 시작 시 턴 생성 서버화

leader만 createTurns를 호출하도록 설계하고, Edge Function 내부에서 최종 검증함으로써 이중 방어 구조로 처리했습니다.

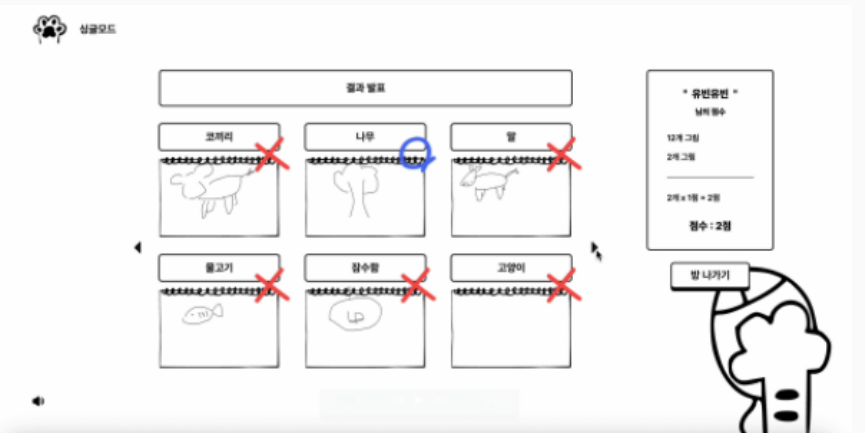
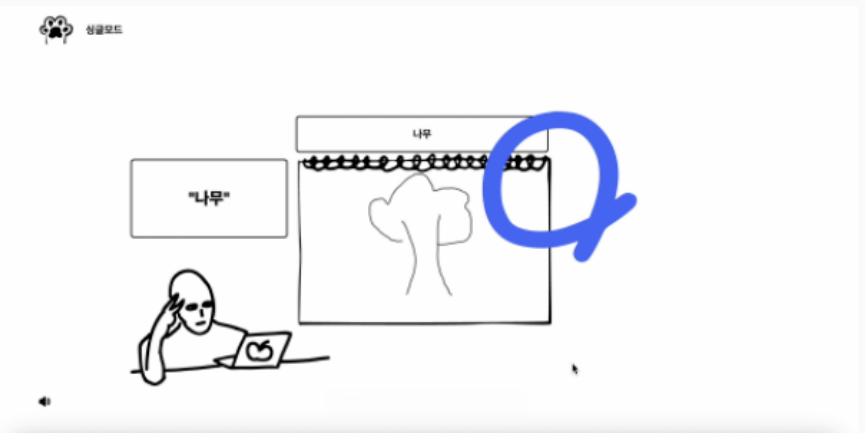
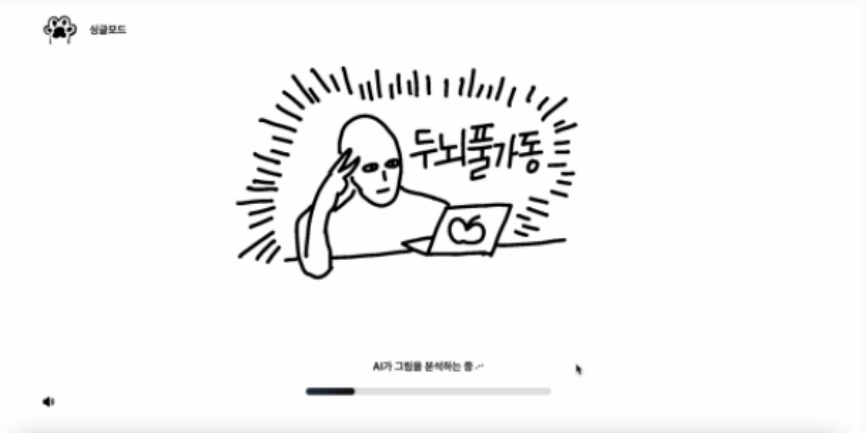
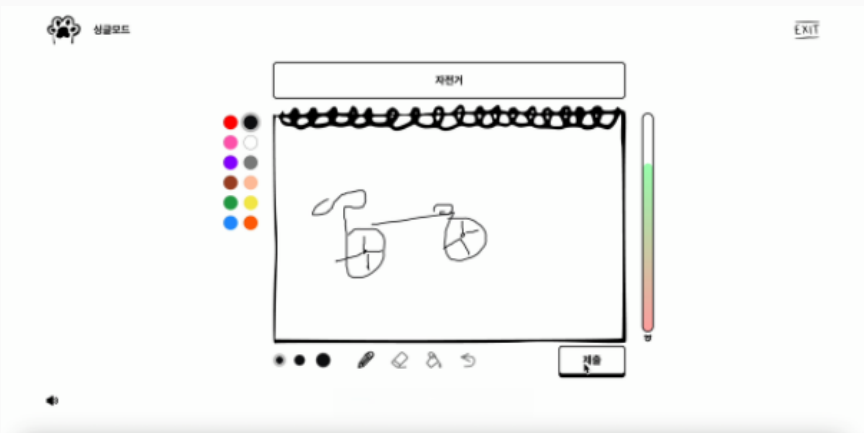
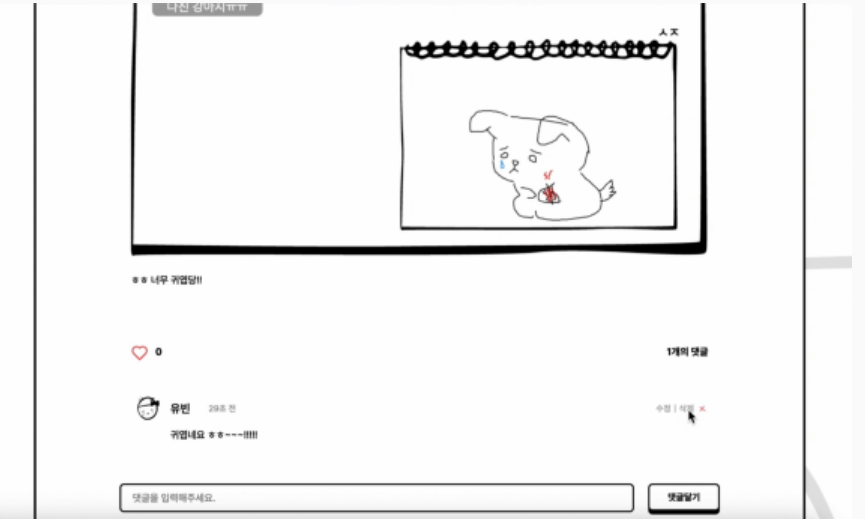
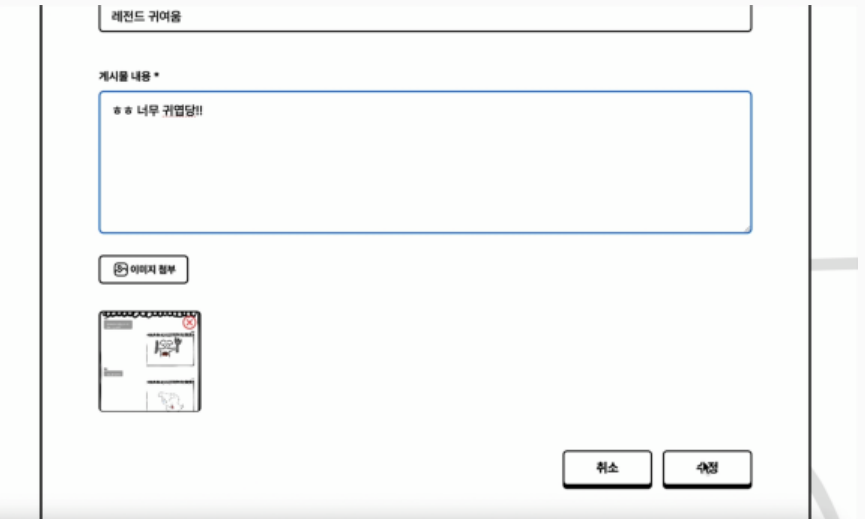
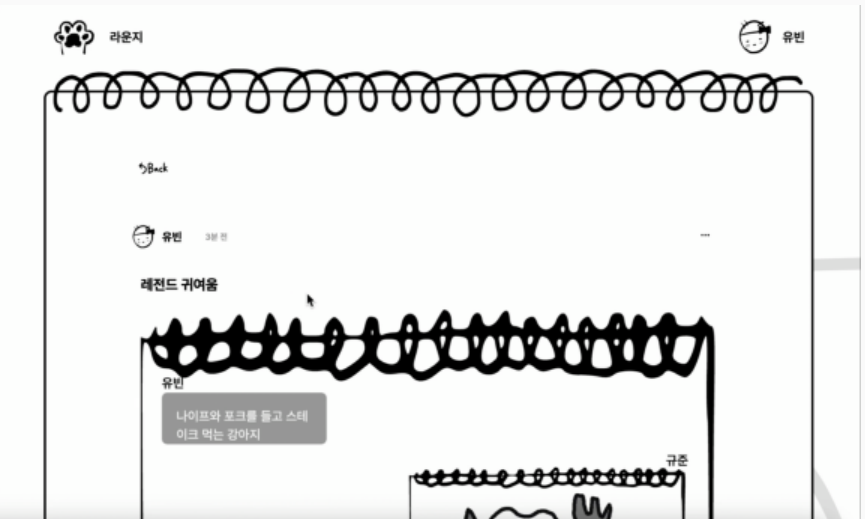
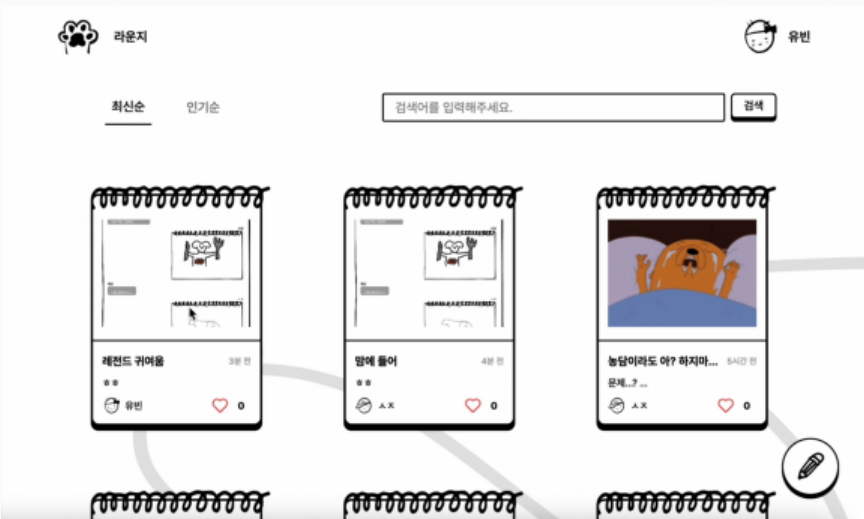
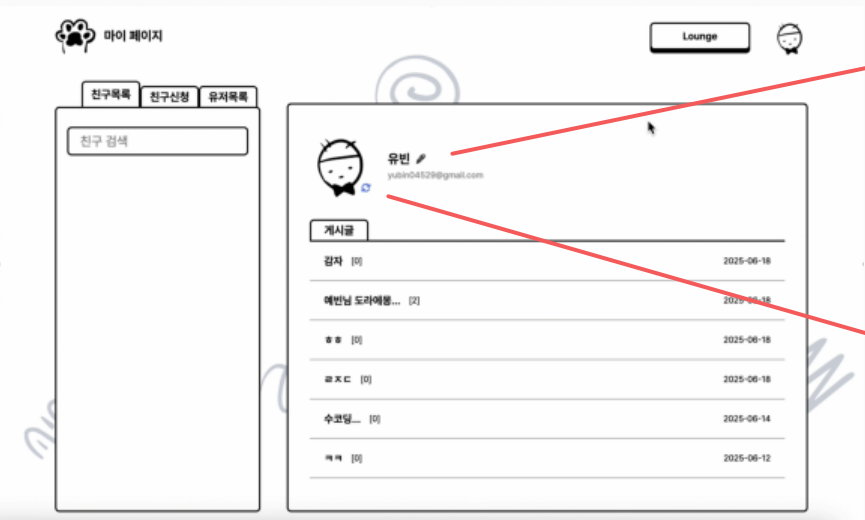
- 전체 플레이어 조회
- 랜덤 셔플
- 자기 자신 배정 방지 구조
- 여러 turn insert 반복 실행
- 조건 검증

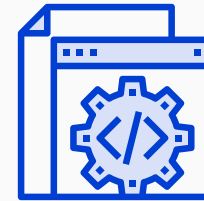
✓ 전체 아키텍처 정리

방 생성과 턴 생성 같은 핵심 로직은 **Supabase Edge Function으로 분리**하여 Service Role 기반 서버 권한으로 처리하고, 클라이언트 조작을 방지했습니다.

특히 턴 생성은 랜덤 셔플과 원형 순환 구조를 활용해 자기 자신 배정을 방지하고, 게임 규칙을 서버에서 강제하도록 설계했습니다.

Project 02 기타 UI





THANK YOU



E-mail : yubin04529@gmail.com



Blog : <https://bbinit.tistory.com>



GitHub : <https://github.com/yubin121>